2

Lab

Lập trình ngôn ngữ Assembly cơ bản

Basic Assembly Programming

Thực hành Lập trình Hệ thống

Lưu hành nội bộ

A. TỔNG QUAN

A.1 Mục tiêu

- Tìm hiểu và làm quen với Hợp ngữ (Assembly Language - ASM)
- Tìm hiểu quá trình dịch từ một mã nguồn assembly thành tập tin thực thi
- Thực hành viết một số chương trình mẫu dưới dạng hợp ngữ (theo AT&T)

A.2 Môi trường

- Máy cài Hệ điều hành Linux (máy ảo).
- Các công cụ biên dịch, hợp dịch, liên kết gồm **gcc**, **as**, **ld**.

A.3 Liên quan

- Sinh viên cần vận dụng kiến thức trong Chương 3 (Lý thuyết).
- Các kiến thức này đã được giới thiệu trong nội dung lý thuyết đã học do đó sẽ không được trình bày lại trong nội dung thực hành này.
- Tham khảo tài liệu (Mục F).

B. KIẾN THỨC NỀN TẢNG

B.1 Hợp ngữ là gì?

Hợp ngữ - Assembly Language (hay viết tắt là ASM) là ngôn ngữ bậc thấp, và nằm ở vị trí trung gian bên cạnh ngôn ngữ máy và ngôn ngữ bậc cao.

ASM sử dụng các từ gợi nhớ (mnemonics) để viết các chỉ thị (instructions) lập trình cho máy tính thay vì bằng những dãy số 0 và 1 (ngôn ngữ máy).

```
push %ebp
mov  %esp, %ebp
sub  $0x8, %esp
movl $0x1, -4(%ebp)
sub  $0xc, %esp
push $0x0
call 8048348
```

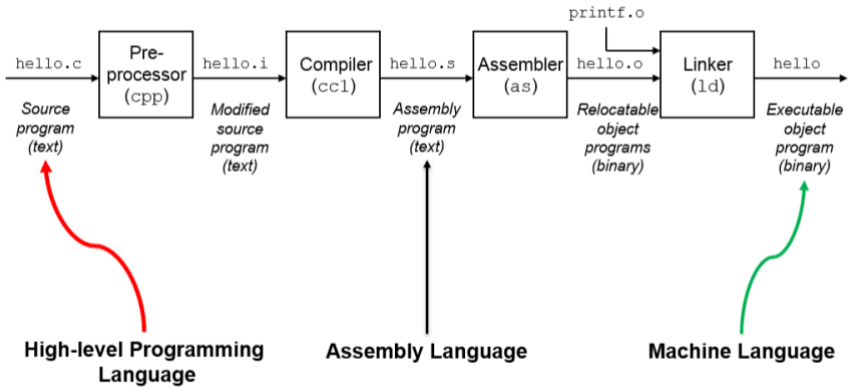
Hình 1. Ví dụ về một đoạn hợp ngữ.

Về **ưu điểm**, Hợp ngữ thân thiện hơn so với ngôn ngữ máy – vốn rất khó hiểu, dễ gây lỗi và tốn nhiều thời giờ. Hợp ngữ có thể tương tác rất sâu dưới hệ thống, chúng có thể giao tiếp trực tiếp với các phần cứng và bắt chúng hoạt động theo ý người lập trình. Bên cạnh đó, hợp ngữ cũng rất hữu ích trong kỹ thuật dịch ngược (reverse engineering) – được sử dụng rộng rãi trong lĩnh vực an toàn thông tin.

Về **nhược điểm**, so với các ngôn ngữ bậc cao quen thuộc (C, C++, Java, Python …), hợp ngữ ít thân thiện hơn, và phụ thuộc vào kiến trúc CPU (ARM, x86-32, x86-64), hệ điều hành (Linux, Windows, Mac) và các tập chỉ thị mà nhà sản xuất phần cứng đưa ra. Nói cách khác, ASM là một ngôn ngữ không giống như các ngôn ngữ lập trình khác, không có một định dạng chuẩn nào cho các trình hợp dịch (Assembler) sử dụng để dịch các chương trình ASM.

Một chương trình viết bằng ngôn ngữ bậc cao hay hợp ngữ không thể được thực thi trực tiếp bởi máy tính. Sau khi được viết xong, chương trình này phải trải qua quá trình dịch thành ngôn ngữ máy. Quá trình này được mô tả trong **Hình 2** bao gồm các giai đoạn:

- Giai đoạn tiền xử lý (Pre-processor)
- Giai đoạn dịch từ ngôn ngữ bậc cao sang hợp ngữ (Compiler)
- Giai đoạn dịch hợp ngữ sang ngôn ngữ máy (Assembler)
- Giai đoạn liên kết (Linker)



Hình 2. Quá trình dịch từ 1 chương trình dạng C sang tập tin thực thi.

Trong bài thực hành này, sinh viên được yêu cầu viết hợp ngữ theo định dạng AT&T (sử dụng Hệ điều hành Linux, các công cụ biên dịch, hợp dịch như as, ld)

B.2 Cấu trúc cơ bản và các thành phần của một chương trình hợp ngữ

B.2.1 Cấu trúc của chương trình hợp ngữ

Trong Linux, các chương trình viết bằng mã assembly sẽ có đuôi **.s**. Thông thường chương trình hợp ngữ có ba phần (section) được khai báo:

B.2.1.1 Section .data

Là section khai báo những **vùng nhớ** hoặc **hằng số**, nơi chứa các dữ liệu dùng cho chương trình. Thường sử dụng để khai báo biến đã được khởi tạo giá trị hoặc các hằng số sử dụng cho chương trình. Định dạng khai báo:

```
.section .data
<name1>:  .<type> <data1>
<name2> = <data2>
...
```

Trong đó:

+ **name** là tên gợi nhớ, có thể được dùng ở các lệnh để truy xuất vùng nhớ hoặc hằng số sau này.
+ **type** là kiểu dữ liệu, ví dụ: int, string, byte...
+ **data** là giá trị cụ thể cần khởi tạo cho vùng nhớ trong vùng nhớ.

Ví dụ: đoạn mã khai báo vùng nhớ **a** lưu số nguyên **10**, vùng nhớ **my_str** lưu chuỗi **"Hello"** và 1 hằng số **b** có giá trị 9.

```
.section .data
a:      .int 10
my_str: .string "Hello"
b = 9
```

B.2.1.2 Section .bss

Phần .bss khai báo các dữ liệu chưa được gán giá trị (hoặc null), thường được dùng khi cần chuẩn bị sẵn các **vùng nhớ** trống để lưu dữ liệu sau này. Đây là thành phần không bắt buộc, nếu không sử dụng có thể bỏ qua. Định dạng khai báo:

```
.section .bss
      .lcomm <name1>, <length>
      .lcomm <name2>, <length>
      ...
```

Trong đó:

+ **name** là tên gợi nhớ, có thể được dùng ở các lệnh để truy xuất vùng nhớ này.

+ **length** là kích thước vùng nhớ cần được cấp, tính theo byte.

Ví dụ: đoạn mã bên dưới khai báo vùng nhớ tên **mem** gồm 2 bytes và vùng nhớ **result** có kích thước 4 bytes.

```
.section .bss
      .lcomm mem, 2
      .lcomm result, 4
```

B.2.1.3 Section .text

Là section **bắt buộc** phải có trong tất cả các chương trình hợp ngữ. Đây là nơi các câu lệnh được khai báo. Định dạng khai báo:

```
.section .text
      .globl _start
_start:
      // các lệnh assembly
```

Sau nhãn _start, các câu lệnh assembly sẽ được viết tùy thuộc vào chức năng cần lập trình. Trong đó có thể sử dụng các vùng nhớ đã được khai báo ở 2 section .data hoặc .bss để lấy hoặc lưu trữ dữ liệu vào các vùng nhớ nhất định, đọc giá trị các hằng số hoặc tương tác với các thanh ghi.

Với 1 vùng nhớ tên **a** được khai báo trong section .data (hoặc .bss):

- Ký hiệu **$a** tương ứng với việc lấy *địa chỉ* của vùng nhớ **a,** được xem là 1 hằng số.
- Ký hiệu **(a)** hoặc **a** tương ứng với việc truy xuất *giá trị* đang lưu tại vùng nhớ **a**.

Với 1 hằng số tên **b** được khai báo trong section .data, ta chỉ có thể lấy giá trị với ký hiệu **$b** trong các lệnh assembly, và ký hiệu này chỉ được đứng ở vị trí source.

B.2.2 Định dạng lệnh assembly

Trong phạm vi bài thực hành này, các lệnh assembly được viết theo định dạng AT&T và tập lệnh của kiến trúc 32bit.

B.3 Linux System Call (Hàm gọi hệ thống Linux)

Có nhiều lời gọi hệ thống (system call) được cung cấp bởi kernel Linux và biết cách tìm cũng như sử dụng chúng rất có lợi trong việc lập trình hợp ngữ (assembly language). Những system call này có sẵn cho các lập trình viên sử dụng. Thông

thường, với mỗi lần phát hành một kernel mới, các system call mới được thêm vào danh sách.

Các system call thường được định nghĩa trong file sau:
/usr/include/asm/unistd.h hoặc */usr/include/asm-generic/unistd.h*

Không giống như các hàm kiểu C, trong đó các giá trị đầu vào được đặt trên stack, các system call yêu cầu đầu vào sao cho các giá trị được đặt trong thanh ghi. Có một thứ tự cụ thể trong đó mỗi giá trị đầu vào được đặt trong các thanh ghi tương ứng. Đặt giá trị đầu vào trong một thanh ghi không đúng chuẩn có thể tạo ra kết quả sai.

Thứ tự trong đó các system call mong đợi các giá trị đầu vào như sau:

- %eax
- %ebx (tham số đầu tiên)
- %ecx (tham số thứ hai)
- %edx (tham số thứ ba)
- %esi (tham số thứ tư)
- %edi (tham số thứ năm)

Sử dụng quy ước này, các giá trị đầu vào sẽ được gán cho các thanh ghi sau:

- %eax: Giá trị của hàm gọi hệ thống (tham khảo bảng bên dưới)
- %ebx: Bộ mô tả tệp (số nguyên)
- %ecx: Con trỏ (địa chỉ bộ nhớ) của chuỗi cần hiển thị
- %edx: Kích thước của chuỗi cần hiển thị

%eax	Name	%ebx	%ecx	%edx	%esx	%edi
1	sys_exit	int	-	-	-	-
2	sys_fork	struct pt_regs	-	-	-	-
3	sys_read	unsigned int	char *	size_t	-	-
4	sys_write	unsigned int	const char *	size_t	-	-
5	sys_open	const char *	int	int	-	-
6	sys_close	unsigned int	-	-	-	-

Giá trị mô tả tệp cho vị trí đầu ra (hoặc vào) để chỉ định vị trí nhận hoặc xuất dữ liệu được đặt trong %ebx. Các hệ thống Linux chứa ba mô tả tệp đặc biệt:

- 0 (STDIN): Đầu vào tiêu chuẩn cho thiết bị đầu cuối (thường là bàn phím)
- 1 (STDOUT): Đầu ra tiêu chuẩn cho thiết bị đầu cuối (thường là màn hình đầu cuối)
- 2 (STDERR): Đầu ra lỗi tiêu chuẩn cho thiết bị đầu cuối (thường là màn hình đầu cuối)

Ví dụ để gọi hàm in ra màn hình chuỗi "Hello, World".

- **%eax = 4** tương ứng với chỉ thị in chuỗi.
- **%ebx = 1** tương ứng với vị trí in ra là stdout (terminal).
- **%ecx = địa chỉ ô nhớ** đang chứa chuỗi cần in.
- **%edx = độ dài** (tính theo byte) của chuỗi cần in.

Sau khi khai báo các giá trị tham số cần thiết của system call trong các thanh ghi, lệnh **int $0x80** được dùng để thực thi system call đó.

```
.section .data
output:
    .string "Hello, World"

.section .text
    .globl _start
_start:
    movl $13, %edx          # message length
    movl $output, %ecx      # address of message to write
    movl $1, %ebx           # file descriptor (stdout)
    movl $4, %eax           # system call number (sys_write)
    int $0x80               # call kernel

    movl $1, %eax           # system call number (sys_exit)
    int $0x80               # call kernel
```

```
lando@ubuntu:~/Test$ as -o example.o example.s
lando@ubuntu:~/Test$ ld -o example example.o
lando@ubuntu:~/Test$ ./example
Hello, World
lando@ubuntu:~/Test$
```

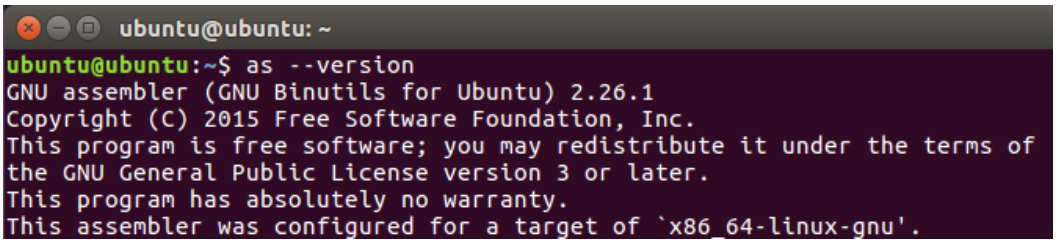
C. THỰC HÀNH

C.1 Yêu cầu 1 – Thiết lập môi trường

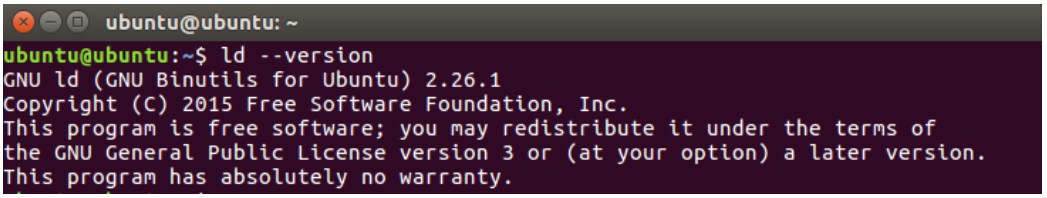
Sinh viên cần chuẩn bị môi trường máy ảo Linux có đầy đủ các công cụ cần thiết bao gồm **as** và **ld** để sử dụng trong quá trình biên dịch file mã hợp ngữ sang file thực thi.

Các công cụ trên có thể được kiểm tra đã được cài đặt trên hệ thống chưa bằng việc gõ các lệnh sau trong terminal:

```
as --version
```




```
ld --version
```



Nếu có kết quả nào trả về không tìm thấy công cụ, sinh viên gõ lệnh để cài đặt:

```
sudo apt-get install binutils
```

C.2 Yêu cầu 2 – Bài tập lập trình hợp ngữ (assembly)

Hướng dẫn chung: Sinh viên thực hiện viết các chương trình dưới dạng hợp ngữ (assembly) trong các file **.s**, sau đó thực hiện các bước hợp dịch và liên kết như sau:

- Hợp dịch với công cụ **as** để thu được một file nhị phân **.o** từ file **.s**:

```
as --32 -o <file .o> <file .s đầu vào>
```

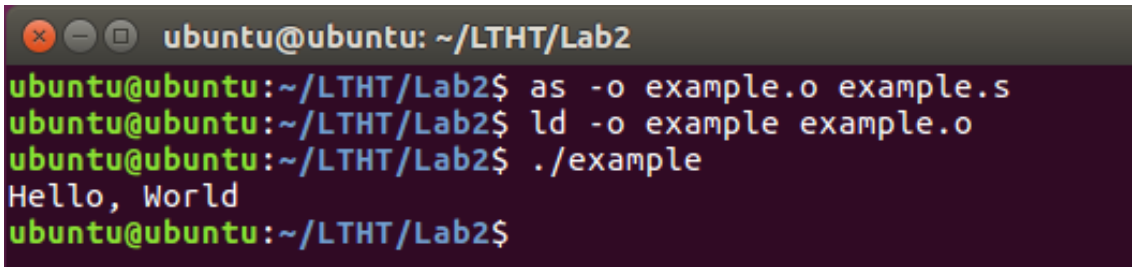
- Liên kết với công cụ **ld** để tạo file thực thi của chương trình

```
ld -m elf_i386 -o <file thực thi> <file .o đầu vào>
```

- Thực thi file

```
./<file thực thi đầu ra>
```

Ví dụ:



Quy ước nhập/xuất (bắt buộc – bài nộp được chấm bằng bộ kiểm thử tự động):

- Chương trình viết theo cú pháp **AT&T, kiến trúc 32-bit**, dùng system call **int $0x80**. Khi hợp dịch/liên kết cần thêm cờ **--32** (as) và **-m elf_i386** (ld), ví dụ: **as --32 -o bai.o bai.s && ld -m elf_i386 -o bai bai.o && ./bai**.
- Mọi input nhập từ bàn phím (stdin), **mỗi giá trị trên một dòng** (kết thúc bằng Enter).
- Khi đọc bằng sys_read (%eax = 3), đặt **%edx = đúng số byte của dòng (gồm cả ký tự xuống dòng)** để không đọc lẫn sang dòng sau.
- In kết quả rõ ràng (một số hoặc một chuỗi kết quả), có thể kèm nhãn.

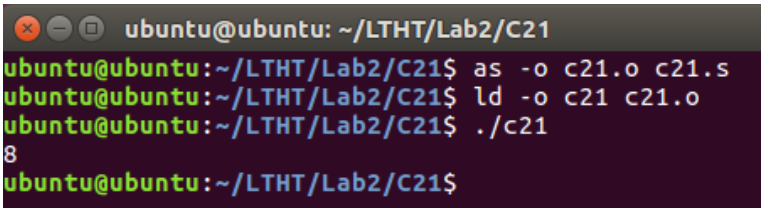
C.2.1 Chương trình in ra độ dài của một chuỗi cho trước (tối đa 9 ký tự) (2đ)

Input: Chuỗi **msg** được khai báo sẵn trong section **.data,** tối đa 9 ký tự.

Output: Xuất ra màn hình độ dài của chuỗi (số ký tự - không tính ký tự null).

Giới hạn: Chuỗi có độ dài tối đa **9 ký tự**

Ví dụ: Với chuỗi **msg = "NT209UIT",** kết quả đầu ra như hình dưới:



Gợi ý 1.1: Phần **D.3** hướng dẫn cách lấy được độ dài của một chuỗi đã khai báo trong section **.data**. *Lưu ý độ dài này sẽ có tính luôn ký tự null ở cuối chuỗi.*

Gợi ý 1.2: Để in một dữ liệu nào đó ra màn hình, cần đảm bảo 2 yếu tố:

- Dữ liệu cần in cần **nằm trong 1 vùng nhớ** (trong section .bss hoặc .data), không hỗ trợ in trực tiếp giá trị từ thanh ghi hay hằng số.
- Khi in, hệ thống tự động xem dữ liệu được cung cấp là **các mã ASCII** của 1 hoặc nhiều ký tự nào đó và in ra các ký tự tương ứng. Do vậy, nếu dữ liệu không phải

là 1 mã ASCII in được thì khi in sẽ không nhìn thấy được. Ví dụ: Khi dữ liệu là 65 thì hệ thống sẽ in ra ký tự 'A', còn dữ liệu là 0 thì không in ra được gì.

Gợi ý 1.3: Ta cần thiết lập giá trị các thanh ghi cho việc xuất dữ liệu như sau:

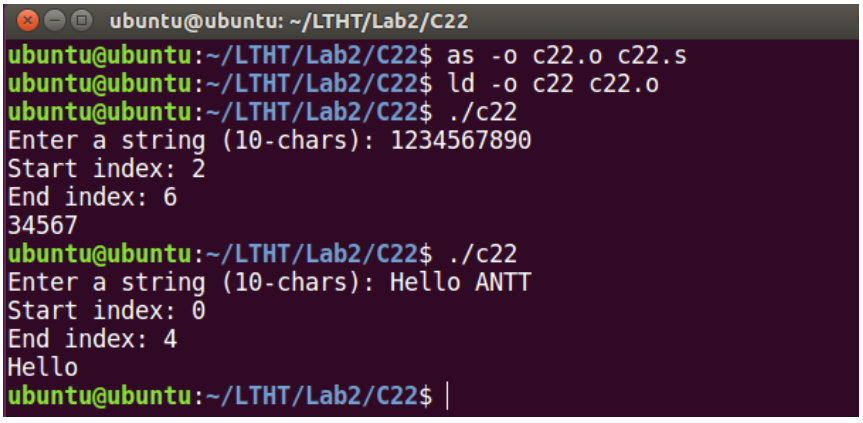
- %eax: SYS_WRITE (4).
- %ebx: với STDOUT (1).
- %ecx: ***địa chỉ ô nhớ*** có dữ liệu cần xuất
- %edx: độ dài sẽ xuất ra (bytes).

C.2.2 Chương trình in ra chuỗi con của một chuỗi 10 ký tự (2đ)

Input: Một chuỗi gồm 10 ký tự bất kỳ (số hoặc chữ không phân biệt hoa - thường) ở dòng 1, chỉ số (index, tính từ 0) start ở dòng 2 và end ở dòng 3 để cắt chuỗi con.

Output: Xuất ra màn hình chuỗi con gồm các ký tự có index từ start đến end (bao gồm cả 2 đầu).

Ví dụ: chuỗi "ABCDEFGHIJ", start = 2, end = 5 → **CDEF**



Gợi ý 2.1: Để nhận input từ bàn phím, ta cần thiết lập giá trị các thanh ghi như sau:

- %eax: SYS_READ (3).
- %ebx: với STDIN (0).
- %ecx: địa chỉ ô nhớ để lưu giá trị nhập vào.

Ta có thể cấp phát sẵn vùng nhớ trong chương trình và truyền địa chỉ của nó cho %ecx. Do chưa gán giá trị nên vùng này nằm trong section **.bss**, kèm theo symbol giống như tên trỏ đến nơi bắt đầu của vùng nhớ.

- %edx: độ dài của chuỗi sẽ nhận – đặt đúng số byte của dòng, gồm cả ký tự xuống dòng (chuỗi 10 ký tự + Enter = 11 byte), để không đọc lẫn sang dòng sau. Vùng nhớ cấp phát phải đủ chứa số byte này.

```
.section .bss
        .lcomm input, 11
```

```
…
    movl $3, %eax
    movl $0, %ebx
    movl $input, %ecx
    movl $11, %edx
    int $0x80
```

Gợi ý 2.2: Dựa vào địa chỉ của chuỗi để truy xuất các ký tự dựa trên chỉ số của chúng. Do 1 ký tự chỉ có kích thước 1 byte nên có thể dùng thanh ghi **%bl** để lấy 1 byte từ một địa chỉ với lệnh **mov**. Ví dụ bên dưới đoạn mã lấy ký tự thứ 2 của chuỗi.

```
    movl $input, %eax
    mov  1(%eax), %bl
```

Cách tham chiếu địa chỉ ô nhớ có thể áp dụng tương tự để lưu chuỗi vào ô nhớ.

Gợi ý 2.3: Input nhập từ bàn phím luôn dưới dạng ký tự, sau khi nhập số cần có bước chuyển từ ký tự số sang giá trị số nguyên tương ứng trước khi tính toán. Xem phần **D2**.

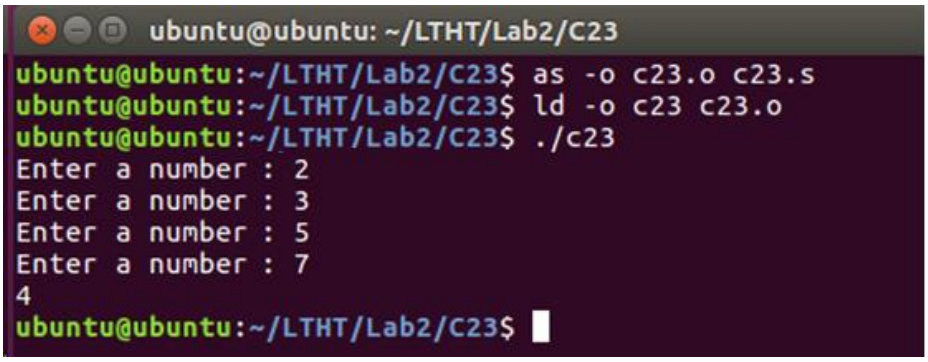
C.2.3 Chương trình tính trung bình cộng của 4 số 1 chữ số (3đ)

Input: 4 số 1 chữ số nhập vào từ bàn phím (mỗi dòng 1 số).

Output: Giá trị trung bình cộng (lấy phần nguyên) của 4 số đã nhập.

Yêu cầu: Các số a, b, c, d nguyên và $0 \le a,b,c,d < 10$.

Ví dụ: 2, 4, 6, 8 → 5; 9, 9, 9, 9 → 9.



Gợi ý 3.1: Sử dụng system call tương tự **C2.2** để nhập dữ liệu.

Gợi ý 3.2: Input nhập từ bàn phím luôn dưới dạng ký tự/chuỗi, cần có bước chuyển từ ký tự số sang giá trị số nguyên tương ứng trước khi tính toán. Xem phần **D2**.

Gợi ý 3.3: Có thể thực hiện chia với lệnh **div** để thực hiện tính trung bình cộng 4 số.

Tham khảo thêm lệnh div: X86-assembly/Instructions/div - aldeid

C.2.4 Mã hóa Caesar cho chuỗi gồm 4 ký tự in thường (3đ)

Input: Chuỗi gồm 4 ký tự chữ in thường, số bước dịch **n** (0 – 9). Chuỗi nhập ở dòng 1, n nhập ở dòng 2.

Output: Xuất ra chuỗi ký tự đã được mã hóa Caesar với bước dịch **n**.

Yêu cầu (bắt buộc): Mỗi ký tự dịch tiến n và xoay vòng trong a..z khi vượt quá 'z'. Ví dụ 'z' + 1 = 'a', 'z' + 2 = 'b'.

Ví dụ (không xoay vòng): "abcd", n = 1 → bcde. Kết quả chạy mẫu:

```
ubuntu@ubuntu: ~/LTHT/Lab2/2024/C24
ubuntu@ubuntu:~/LTHT/Lab2/2024/C24$ as -o c24.o c24.s
ubuntu@ubuntu:~/LTHT/Lab2/2024/C24$ ld -o c24 c24.o
ubuntu@ubuntu:~/LTHT/Lab2/2024/C24$ ./c24
Nhap chuoi (4 ky tu): abcd
Nhap n (0-9): 1
bcde
ubuntu@ubuntu:~/LTHT/Lab2/2024/C24$ ./c24
Nhap chuoi (4 ky tu): abcd
Nhap n (0-9): 5
fghi
ubuntu@ubuntu:~/LTHT/Lab2/2024/C24$
```

Ví dụ (có xoay vòng): "wxyz", n = 3 → zabc. Kết quả chạy mẫu:

```
ubuntu@ubuntu: ~/LTHT/Lab2/2024/C24
ubuntu@ubuntu:~/LTHT/Lab2/2024/C24$ as -o c24.o c24.s
ubuntu@ubuntu:~/LTHT/Lab2/2024/C24$ ld -o c24 c24.o
ubuntu@ubuntu:~/LTHT/Lab2/2024/C24$ ./c24
Nhap chuoi (4 ky tu): abyz
Nhap n (0-9): 1
bcza
ubuntu@ubuntu:~/LTHT/Lab2/2024/C24$ ./c24
Nhap chuoi (4 ky tu): name
Nhap n (0-9): 9
wjvn
ubuntu@ubuntu:~/LTHT/Lab2/2024/C24$
```

D. THAM KHẢO

D.1 Một số lưu ý khi lập trình assembly định dạng AT&T

- **Cần phân biệt được các toán hạng:**

Khác biệt trong ký hiệu của các dạng toán dạng khác nhau:

- Hằng số: **$1, $0x23, $'B'...**
- Thanh ghi: **%eax, %al,...**
- Địa chỉ ô nhớ: **$output** với output là nhãn của ô nhớ trong .data hoặc .bss
- Giá trị trong ô nhớ: **(output)** hoặc **output**. Số lượng byte được lấy phụ thuộc vào sufix của lệnh assembly.

- **Trong các câu lệnh assembly, cần đảm bảo:**
  - Có **tối đa 1 toán hạng là hằng số**, nếu có hằng số cần đứng ở vị trí src.
    Ví dụ: cmpl $0, %eax
  - Có **tối đa 1 toán hạng liên quan đến truy xuất ô nhớ**.

- **Cách viết biểu thức địa chỉ để truy xuất ô nhớ**

Giả sử với ô nhớ có nhãn tên **output**

- Truy xuất giá trị ô nhớ từ địa chỉ bắt đầu: viết dưới dạng **(output)** hoặc **output**.
- Truy xuất giá trị ô nhớ từ địa chỉ cách n bytes: cần đưa địa chỉ vào thanh ghi và dùng biểu thức tính toán địa chỉ

```
movl $output, %eax
movl 1(%eax), %ebx    // truy xuất từ địa chỉ cách 1 byte
```

D.2 Bảng mã ASCII

- **Chuyển đổi số có 1 chữ số**

Sử dụng bảng mã ASCII để chuyển đổi từ số (decimal) sang chữ (dạng ascii) và ngược lại. Tham khảo tại https://www.ascii-code.com/. Ví dụ: Có thể chuyển đổi từ số 5 sang ký tự '5' bằng cách cộng **$48** (ký tự '0'). Ngược lại, để chuyển từ ký tự sang số thì trừ ký tự '0'.

- **Chuyển đổi số có nhiều chữ số**

Tách riêng từng số (hay ký tự số) sau đó áp dụng cách chuyển đổi trên số có 1 chữ số để chuyển qua lại giữa số và ký tự số tương ứng của nó.

Ví dụ: Chuyển chuỗi ký tự số '123' sang số nguyên: 1x100 + 2x10 + 3 = 123.

D.3 Lấy độ dài chuỗi trong section .data

Như đã trình bày ở trên, trong section .data có thể dùng để khai báo một số biến đã gán trước giá trị hoặc hằng số được dùng trong chương trình, ví dụ chuỗi output sẽ in ra màn hình. Tuy nhiên, thay vì gán cứng độ dài các chuỗi này khi dùng trong các system call, đoạn khai báo bên dưới cho phép lấy giá trị độ dài của chuỗi **rs**, lưu ý dù nằm trong .data nhưng **len** là hằng số/biến.

Để lấy được giá trị số nguyên độ dài của một chuỗi, có thể khai báo một biến hoặc hằng số **len** **ngay sau** chuỗi cần quan tâm, và tính ra độ dài bằng cách lấy địa chỉ của biến/hằng số vừa tạo trừ đi địa chỉ bắt đầu của chuỗi đã cho.

Minh hoạ tương quan giữa độ dài và cách lưu trữ chuỗi:

Chuỗi cho trước | Biến **len**

N	T	2	0	9	U	I	T	\0	0	0	0	0
100	101	102	103	104	105	106	107	108	109	10A	10B	10C

Minh hoạ cách lưu trữ của chuỗi.

Độ dài của chuỗi (tính cả ký tự null) = 109 – 100 (Địa chỉ của len – địa chỉ của chuỗi).

```
.section .data
rs:  .string "Nt209UIT"
len: .int 0
…
    movl $len, %edi # $len là địa chỉ của len
    subl $rs, %edi  # $rs là địa chỉ của rs
    subl $1, %edi   # Trừ 1 ký tự null
    movl %edi, len  # Độ dài của rs đã lưu vào len
```

hoặc

```
.section .data
rs:  .string "Max is "
rs_len = . -rs
```

E. YÊU CẦU & ĐÁNH GIÁ

Sinh viên thực hành theo **nhóm tối đa 3 sinh viên**, với cách đánh giá như sau:

- Phần trên lớp: demo kết quả với GVTH.
- Nộp báo cáo: nộp các file code .s tại website môn học theo thời gian quy định, *có chú thích chức năng của các đoạn code.*
- Thang điểm Lab 2: **10đ** (C21: 2đ, C22: 2đ, C23: 3đ, C24: 3đ). Bài được chấm bằng bộ kiểm thử tự động gồm nhiều ca, kể cả ca biên – cần tuân thủ đúng quy ước nhập/xuất ở phần Hướng dẫn chung.

Tạo **thư mục** tên **Lab2-NhomX-MSSV1-MSSV2-MSSV3** và nộp **các file .s** với định dạng:

Lab2-NhomX-<yêu cầu>.s

Ví dụ: *Lab2-Nhom2-C21.s*

F. THAM KHẢO

[1] Linux assemblers: A comparison of GAS and NASM [Online] Available at: https://www.ibm.com/developerworks/library/l-gas-nasm/index.html

[2] Randal E. Bryant, David R. O'Hallaron (2011). *Computer System: A Programmer's Perspective (CSAPP)*

[3] Richard Blum (2005). *Professional Assembly Language*

HẾT